

Manual Paso a Paso - Aplicación de Autenticación Full Stack

Este documento resume el proceso completo de construcción de la aplicación de autenticación, explicando no solo qué se hizo, sino por qué se hizo cada paso.

1. Crear el Backend

Se creó un proyecto Node.js con Express y TypeScript. El objetivo era construir una API que gestionara usuarios, autenticación y autorización.

2. Configurar PostgreSQL

Se creó una base de datos llamada auth_system y una tabla users para almacenar usuarios, contraseñas cifradas y roles.

3. Crear Registro de Usuarios

Se implementó POST /api/auth/register para recibir datos, validarlos, cifrar la contraseña con bcrypt y guardar el usuario.

4. Crear Login

Se implementó POST /api/auth/login para verificar credenciales y generar un JWT que representara la sesión del usuario.

5. Implementar JWT

JWT permitió identificar usuarios autenticados sin almacenar sesiones en el servidor.

6. Crear Auth Middleware

Se creó un middleware que lee el token Bearer, lo verifica y adjunta la información del usuario a la petición.

7. Crear Roles

Se añadió el campo role para diferenciar usuarios normales y administradores.

8. Crear Middleware Admin

Se verificó que solo usuarios con rol admin pudieran acceder a ciertas rutas.

9. Crear Ruta Profile

Permitió obtener información del usuario autenticado usando el token.

10. Construir Frontend React

Se creó la interfaz usando React, TypeScript y React Router.

11. Crear AuthContext

Se centralizó la autenticación en un contexto global para compartir usuario y token entre componentes.

12. Guardar Sesión

Se utilizó localStorage para mantener la sesión activa después de recargar la página.

13. Crear ProtectedRoute

Se protegieron rutas privadas para impedir acceso sin autenticación.

14. Crear GuestRoute

Se evitó que usuarios autenticados volvieran a login o register.

15. Crear Panel Admin

Se construyó una vista para listar usuarios y gestionar permisos.

16. Cambio de Roles

Se implementó PUT /api/users/:id/role para promover o degradar usuarios.

17. Eliminar Usuarios

Se implementó DELETE /api/users/:id para eliminar registros.

18. Seguridad Adicional

Se evitó que un administrador se eliminara a sí mismo o perdiera accidentalmente su propio rol.

19. Pruebas con Postman

Se verificaron registros, login, tokens JWT, permisos, errores 401 y 403.

20. Migración a Neon

Se reemplazó PostgreSQL local por Neon usando DATABASE_URL.

21. Resolver dotenv

Se detectó que process.env era undefined y se solucionó cargando dotenv correctamente.

22. Preparar Producción

Se añadieron scripts build y start para compilar TypeScript y ejecutar el backend.

23. Deploy Backend

Se desplegó Express en Render y se configuraron las variables DATABASE_URL y JWT_SECRET.

24. Deploy Frontend

Se desplegó React en Vercel y se configuró VITE_API_URL.

25. Solucionar React Router

Se agregó vercel.json para evitar errores 404 al recargar rutas internas.

26. Resultado Final

Se obtuvo una aplicación Full Stack con React, Express, PostgreSQL, JWT, roles, permisos y despliegue completo en la nube.